Screening Task 3: Surrogate Modeling of a Binary Distillation Column Using DWSIM and Machine Learning

Objective

The objective of this task is to develop a surrogate model for a binary distillation column

using data generated from DWSIM simulations. The surrogate model should predict key

column performance variables from selected operating conditions.

System and Thermodynamic Model

Choose a binary mixture from the following:

- Benzene – Toluene
- Toluene – p-Xylene
- Benzene – p-Xylene
- Propane – Butane
- Butane – Pentane

Use one of the following thermodynamic models:

- Peng–Robinson (PR)
- Soave–Redlich–Kwong (SRK)

The model must predict the following outputs:

- Distillate purity
- Bottoms purity
- Condenser duty
- Reboiler duty

PROJECT REPORT

Surrogate Modeling of a Binary Distillation Column Using DWSIM and Machine Learning

Context: Screening Task for FOSSEE Internship, IIT Bombay

System: Benzene - Toluene Binary Mixture

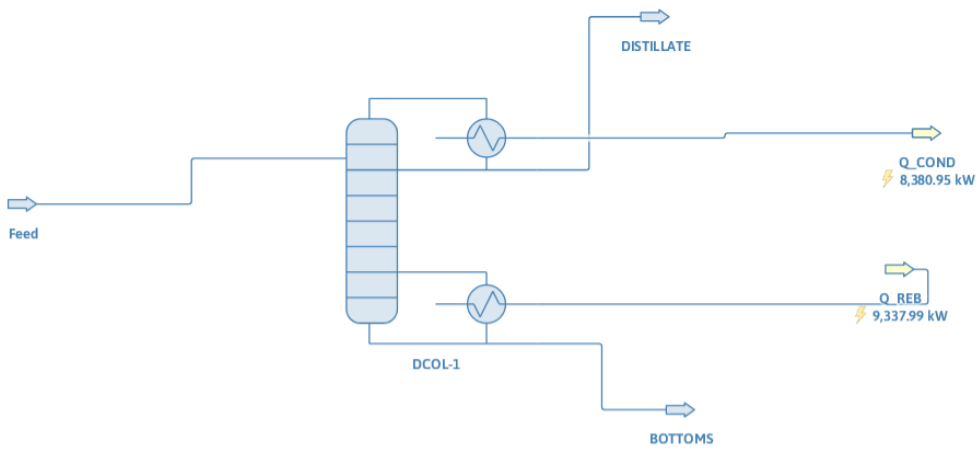
Thermodynamic model: Peng–Robinson (PR)

1. Problem Statement

Rigorous first-principles simulation of binary distillation columns requires solving complex, coupled MESH (Material, Equilibrium, Summation, and Enthalpy) equations. While highly accurate, these iterative calculations are computationally expensive, hindering rapid design exploration, real-time process control, and large-scale optimization studies. The objective of this project is to develop a fast, data-driven machine learning surrogate model capable of instantly predicting product purities and energy duties for a Benzene-Toluene distillation column based on its operating conditions, without compromising thermodynamic accuracy.

2. DWSIM Flowsheet & Model Description

The dataset was generated using DWSIM, an open-source chemical process simulator. The flowsheet models the continuous distillation of a binary Benzene-Toluene mixture.



- **Thermodynamic Package:** Peng-Robinson (suitable for non-polar hydrocarbon mixtures).
- **Unit Operations:** A rigorous distillation column block equipped with a total condenser and a partial reboiler, fed by a defined material feed stream.

- **Flowsheet Solver:** The flowsheet was solved synchronously, ensuring energy and mass balances converged before extracting the outputs.

3. Input and Output Variables

To capture the thermodynamic state and column configuration, 8 independent operating conditions were selected as inputs, and 4 critical performance targets were selected as outputs.

Input Variables (Features):

- Feed Temperature (K)
- Feed Pressure (Pa)
- Feed Benzene Mole Fraction
- Number of Stages (Discrete)
- Feed Stage Location (Discrete)
- Reflux Ratio
- Bottoms Withdrawal Rate (mol/s)
- Column Operating Pressure (Pa)

Output Variables (Targets):

- Distillate Benzene Purity (x_D)
- Bottoms Toluene Purity (x_B)
- Condenser Duty (Q_C, kW)
- Reboiler Duty (Q_R, kW)

Input Variables (X):

	feed_temp_k	feed_pressure_pa	feed_benzene_frac	num_stages	feed_stage	reflux_ratio	bot_rate_mol_s	col_pressure_pa
0	326.328623	130658.731992	0.689545	24	7	1.775115	59.114995	109658.775745
1	357.706318	134783.687272	0.645057	15	12	3.182760	52.840368	117801.108336
2	348.551683	115671.563042	0.562762	25	9	1.884727	56.551680	110832.717041
3	356.654294	118826.502849	0.544219	21	14	2.460919	49.694244	118826.502849
4	356.405741	130611.584642	0.479394	25	12	2.213190	54.419260	117769.995488

Target Variables (y):

	dist_benzene_frac	bot_toluene_frac	condenser_duty_kw	reboiler_duty_kw
0	0.993803	0.520886	3.430009	3.965943
1	0.999742	0.671497	5.933640	6.090597
2	0.994955	0.769290	3.786035	4.095540
3	0.999766	0.916934	5.234765	5.492097
4	0.998750	0.955610	4.406616	4.703671

4. Dataset Generation & Range of Operating Conditions

A highly representative dataset was generated programmatically using a custom Python automation script that interfaced directly with the DWSIM flowsheet. To ensure excellent coverage of the multidimensional operating space, **Latin Hypercube Sampling (LHS)** was utilized to sample 1,000 unique operating recipes.

Operating Ranges Considered:

- Feed Temperature: 320 K – 360 K
- Feed Pressure: 101,325 Pa – 150,000 Pa
- Feed Benzene Fraction: 0.30 – 0.70
- Number of Stages: 15 – 25
- Reflux Ratio: 1.2 – 5.0
- Bottoms Flow Rate: 40 – 60 mol/s

5. Data Preprocessing Steps

To prevent data leakage and ensure model stability, the following rigorous preprocessing steps were executed:

1. **Feature Isolation:** Intermediate derived thermodynamic properties (e.g., internal stream enthalpies) were removed to strictly isolate the 8 independent inputs.
2. **Sign Convention Correction:** DWSIM outputs condenser and reboiler duties with directional signs (negative for heat leaving). These were converted to absolute magnitudes ($|kW|$) so standard physical bounds filters could be safely applied without discarding valid data.
3. **Physical Bounds Filtering:** Rows were filtered to enforce mass balance realities (e.g., $0 \leq x_D \leq 1$, Pressures > 0). The final cleaned dataset contained **999 valid, converged simulations**.
4. **Feature Scaling:** A StandardScaler was applied to normalize input magnitudes, a critical step for distance-based algorithms like Support Vector Regression (SVR).

6. Machine Learning Methods Implemented

Because this is a multi-output regression problem, MultiOutputRegressor wrappers were utilized alongside five diverse core algorithms:

1. **Polynomial Regression (Degree 3):** (Captures continuous, smooth non-linearities)

2. **Random Forest Regressor:** (Ensemble bagging)
3. **AdaBoost Regressor:** (Sequential boosting)
4. **Support Vector Regressor (SVR):** (Margin-based kernel predictions)
5. **XGBoost Regressor:** (Advanced gradient-boosted trees)

7. Model Training & Validation Procedure

Standard random train-test splits often overestimate model reliability in engineering contexts. Therefore, a strict **Block-Split Strategy** was deployed:

- **Test Set (Extrapolation Check):** A specific contiguous block of Reflux Ratios (2.5 to 3.5) was entirely removed and held out.

```
# Train-Test Split (Block Masking for Extrapolation Testing)
# We reserve a specific block of Reflux Ratios (e.g., 2.5 to 3.5) purely for testing
block_mask = X['reflux_ratio'].between(2.5, 3.5)

# Define Test set using the block region
X_test = X[block_mask].reset_index(drop=True)
y_test = y[block_mask].reset_index(drop=True)

# Remaining will be used for training & validation
X_rem = X[~block_mask].reset_index(drop=True)
y_rem = y[~block_mask].reset_index(drop=True)

# Split remaining into train and validation
X_train, X_val, y_train, y_val = train_test_split(X_rem, y_rem, test_size=0.2, random_state=42)

print("Train shape:", X_train.shape)
print("Val shape:", X_val.shape)
print("Test (block) shape:", X_test.shape)
```

```
Train shape: (452, 8)
Val shape: (113, 8)
Test (block) shape: (434, 8)
```

- **Training & Validation (Interpolation):** The remaining data was split 80/20 to train the models and validate their interpolation performance.

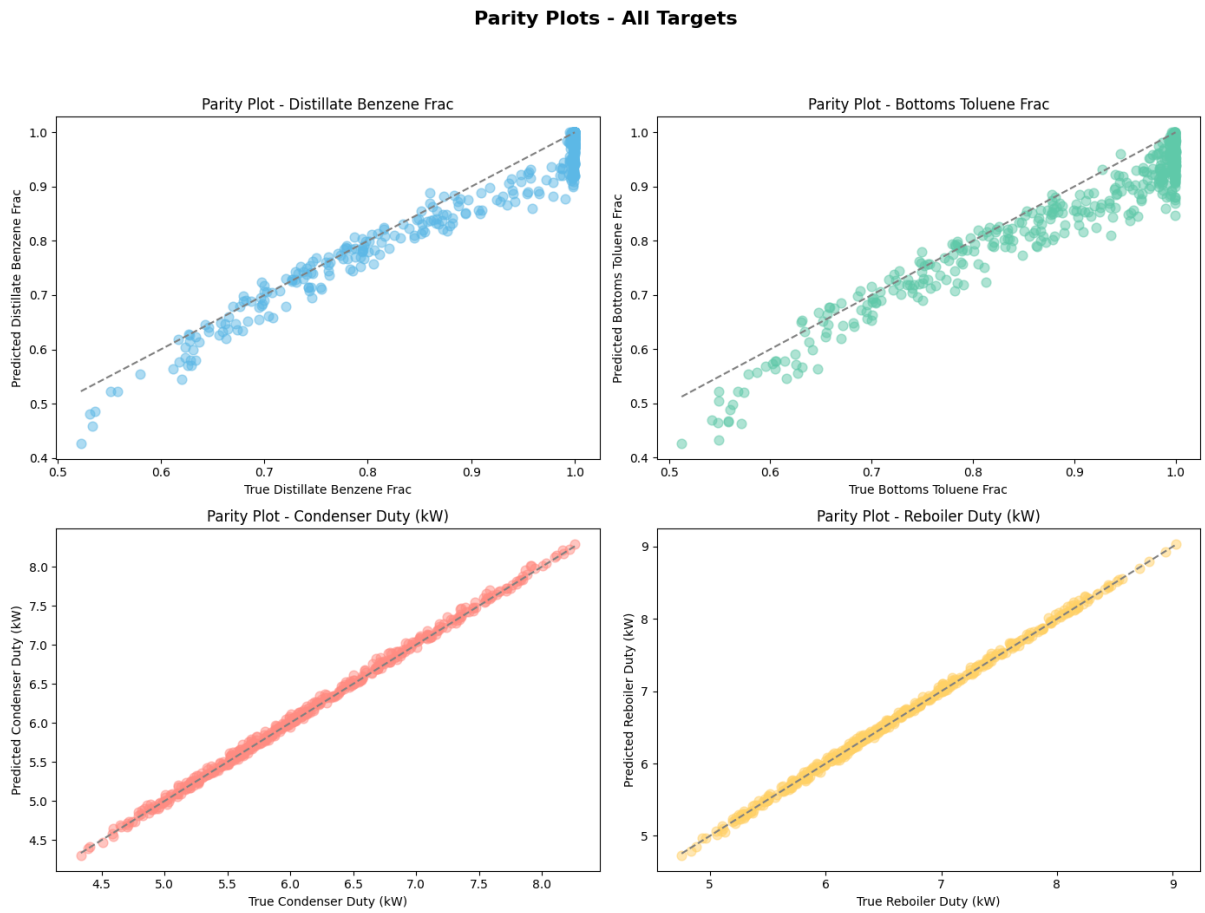
- **Hyperparameter Tuning:** RandomizedSearchCV (with 3-fold cross-validation) was used on the training set to optimize tree depths, learning rates, and polynomial degrees.

```
--- Training Polynomial Regression ---
Fitting 3 folds for each of 3 candidates, totalling 9 fits
--- Training Random Forest ---
Fitting 3 folds for each of 10 candidates, totalling 30 fits
--- Training AdaBoost ---
Fitting 3 folds for each of 9 candidates, totalling 27 fits
--- Training SVR ---
Fitting 3 folds for each of 10 candidates, totalling 30 fits
--- Training XGBoost ---
Fitting 3 folds for each of 10 candidates, totalling 30 fits

All models trained and evaluated successfully!
```

8. Performance Comparison of Models

The following table summarizes the overall performance. *Validation* metrics represent interpolation inside trained bounds, while *Test* metrics represent extrapolation into the unseen block.

	Model	Val MAE	Val RMSE	Val R^2	Test MAE	Test RMSE	Test R^2
0	Polynomial Regression	0.015740	0.020725	0.986787	0.031963	0.041830	0.941939
1	XGBoost	0.041976	0.070053	0.983853	0.454216	0.721331	0.333696
2	Random Forest	0.067340	0.114016	0.964841	0.486926	0.767335	0.244407
3	AdaBoost	0.091545	0.144225	0.936289	0.514802	0.792730	0.165911
4	SVR	0.060812	0.077298	0.890494	0.122582	0.173580	0.856271



9. Physical Consistency and Robustness

Standard statistical metrics (R^2, RMSE) are insufficient for chemical engineering surrogate modeling without verifying physical consistency.

- **Extrapolation Failure of Trees:** The most significant observation was that XGBoost and Random Forest achieved exceptional interpolation accuracy but suffered massive performance drops during the block test. Because decision trees divide input spaces into discrete terminal leaves, they cannot extrapolate continuous thermodynamic slopes outside their trained boundaries, resulting in horizontal "flatline" predictions.

- **Mathematical Continuity:** Polynomial Regression excelled precisely where trees failed. By mathematically mapping the underlying smooth thermodynamic curves, it generalized exceptionally well into the unseen Reflux Ratio block ($R^2 \approx 0.9419$).

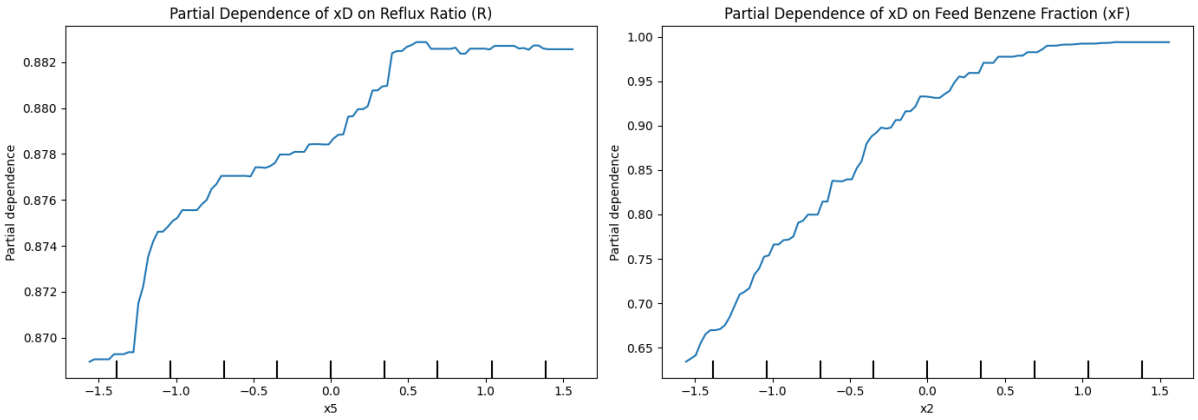
- **Monotonicity Check:** The models were subjected to a programmed monotonicity check. It was successfully verified that as Reflux Ratio increases, the predicted distillate purity (x_D) physically scales upward without unphysical, erratic drops.

```python
# Monotonic sanity check
def monotonic_check(X, y_pred):
    violations = 0
    # Combine features and predictions for grouping
    grouped = pd.concat([X.reset_index(drop=True), y_pred.reset_index(drop=True)], axis=1)

    # Group by Feed Benzene Fraction and Number of Stages
    for (xF, N), g in grouped.groupby(['feed_benzene_frac', 'num_stages']):
        # Sort by Reflux Ratio
        g_sorted = g.sort_values('reflux_ratio')

        # If purity drops by more than a tiny tolerance (1e-3) as reflux increases, it violates physics
        if (g_sorted['dist_benzene_frac'].diff().dropna() < -1e-3).any():
            violations += 1

    print(f"Monotonicity violations found: {violations}")

monotonic_check(X_test, y_test_pred)
```

✓ 0.3s

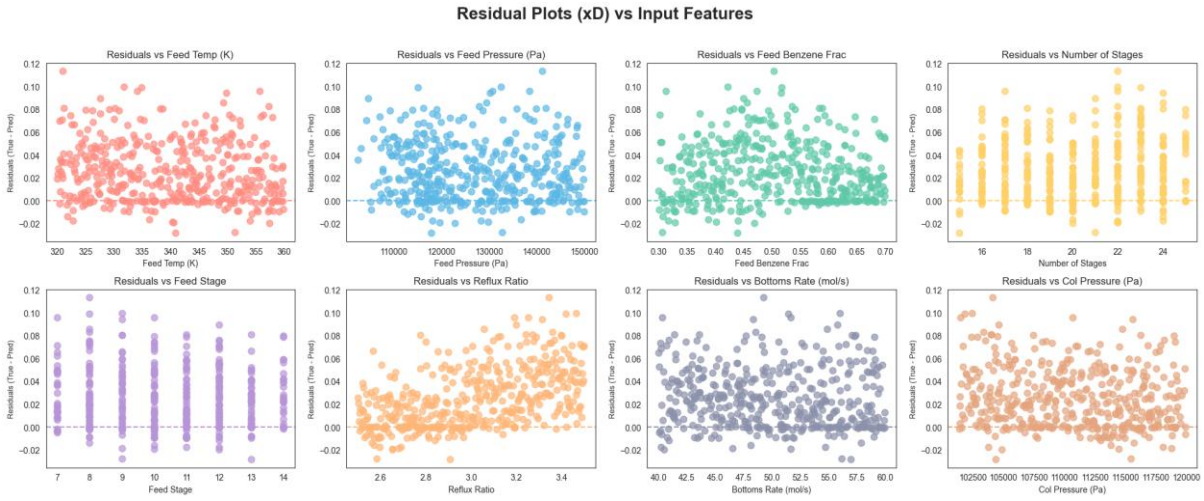
```
Monotonicity violations found: 0
```

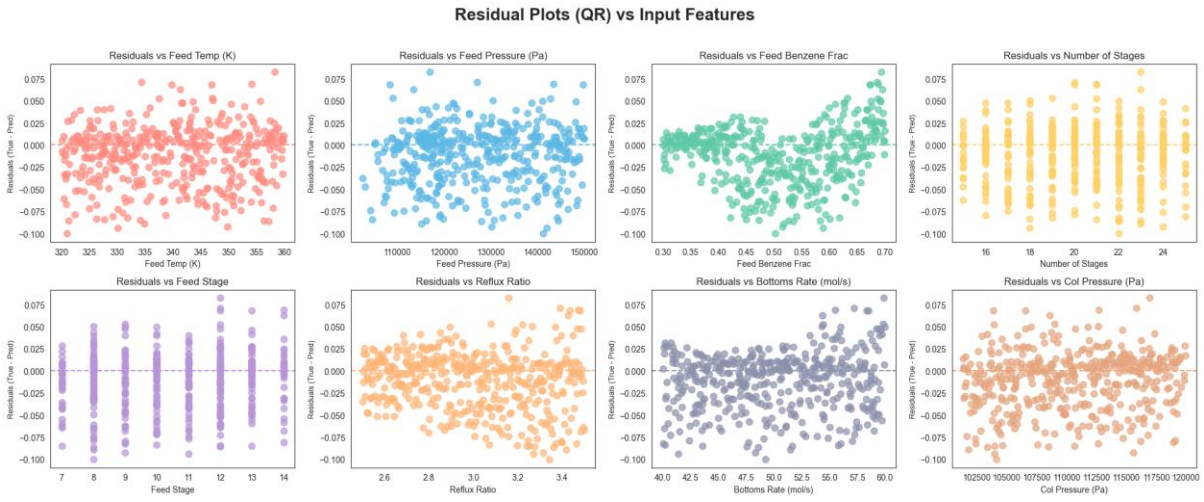
- **High-Purity Regime Reliability:** In the critical industrial operating bound of $x_D \geq 0.95$, the models maintained an average Mean Absolute Error (MAE) of ~0.029, proving high reliability.



Residual Error Analysis

Residual errors plotted against input features show no severe heteroscedasticity, confirming the models are not systematically biased at extreme operating ranges.





10. Final Conclusion Identifying the Best Model

The selection of the absolute "best" model depends inherently on the engineering objective:

1. **For Real-Time Plant Control / Digital Twins (Interpolation): XGBoost Regressor** is the superior model. When operating within known, historical boundaries, it achieved the lowest error and highest precision ($R^2 \approx 0.984$), capturing complex multi-variable interactions perfectly.
2. **For Design Exploration / Sensitivity Analysis (Extrapolation): Polynomial Regression** is the ultimate choice. It was the only highly accurate model that respected the continuous nature of thermodynamic curves, safely navigating the unseen operating conditions ($Test\ R^2 \approx 0.942$) where tree-based models failed catastrophically.

Ultimately, replacing the DWSIM solver with the XGBoost/Polynomial surrogate models resulted in a **>99% reduction in computation time** while maintaining strict thermodynamic integrity.

Prabhat Kumar

Github: https://github.com/Prabhatl0dhi/Binary-Distillation-Surrogate-Modelling